RecoverIQ: Software Requirements Specification

Oct 31, 2026 · @BIT Project -PS team

RecoverIQ is an AI agent that detects an emerging customer-experience problem, proves its root cause against operational evidence, finds the customers hurt, and drives an approved, measured recovery. This document states every requirement, how each is checked, and what it gives the customer and the business.

1. Introduction

This SRS defines RecoverIQ, the team's answer to TCS Challenge 06, Autonomous Customer Experience Recovery.

1.1 Purpose

It gives the build team, the frontend teammate and the jury one agreed statement of what the system does, what it must never do, and how each claim is measured.

1.2 Scope

In scope:

- Ingestion of feedback (API, CSV, simulated platform imports) and operational events (payments, orders, deliveries, releases).
- An AI agent that runs the chain Signal → Issue → Segment → Root Cause → Impact → Intervention.
- A Root Cause Court that tests every hypothesis against evidence.
- Human approval, simulated recovery actions and KPI-verified closure.
- Adaptive reassessment when new evidence arrives, and a self-evaluation scorecard against hidden truth.

Out of scope: real refunds, real customer messages, real production changes, and live connections to third-party platforms.

1.3 Document control

Item	Value
Version	1.0
Date	31 Oct 2026
Event	TCS Technology Day 2026 AI Hackathon, BIT
Challenge	06, Autonomous Customer Experience Recovery (Theme 2, AI for Business)
Services	Node.js backend on port 5000, Python AI service on port 8000, React client on 5173

1.4 How requirements are written

Each requirement has an ID, a priority and the benefit it delivers. Prefixes: FR (functional), DR (data), AR (AI and LLM), IR (interface), NFR (non-functional), AT (acceptance test).

- **Must:** the golden path fails without it.
- **Should:** cut only if time runs out.
- **Could:** first to cut.

2. Problem and stakeholders

Customer evidence is spread across reviews, tickets, social posts, transcripts, transactions and telemetry, and sentiment analysis says customers are unhappy without saying what the business should change first.

2.1 What Challenge 06 asks for

PS wording	What the system must do
Signal → Issue → Segment → Root Cause → Impact → Intervention	Run this chain as one connected investigation, each step building on the last
Combine multiple feedback channels	Join reviews, tickets, social, transcripts and operational events into one view

PS wording	What the system must do
Cluster emerging issues	Find new problems without predefined categories
Estimate severity	Give severity as a number with its inputs
Identify impacted segments	Compare issue rates by segment against their population, not raw counts
Correlate operational or product changes	Line issue onset up against releases, outages and policy changes
Evaluate alternative explanations	Test and reject decoy causes with evidence
Recommend interventions; define outcome KPIs	Ranked, sequenced actions, each with a measurable KPI
Guardrail: sentiment alone is not proof	Block sentiment from counting as root-cause evidence, in code

TCS also expects at least 5 of 7 capabilities (reason across sources, plan, act through tools, verify, adapt, human control, evaluate with metrics) and an unseen jury scenario. RecoverIQ targets all 7.

2.2 Users

User	What they do	What they need
CX or operations manager	Reviews incidents, approves or rejects actions	Evidence they can trust and defend, control over risky actions
Support lead	Uses scripts and recovery lists	Clear guidance on what to tell affected customers
Customer (end user of the food app)	Receives the recovery outcome	Money back, an honest message, the problem fixed
Jury	Watches the demo and injects a scenario	Visible reasoning, measured results, adaptation

3. Solution overview

RecoverIQ keeps one persistent incident case from the first signal to verified recovery, and every step writes its evidence into that case.

3.1 The incident case loop

1. **Detect.** Statistics flag an unusual rise in an issue across channels.
2. **Plan.** The agent writes an investigation plan for what evidence is missing.
3. **Investigate.** It calls tools: clustering, segment lift, change-log lookup, telemetry queries.
4. **Root Cause Court.** Each hypothesis faces five tests; sentiment cannot count as proof.
5. **Impact.** Affected customers, money at risk and orders lost, measured from transactions.
6. **Recommend.** Interventions ranked and sequenced, with eligibility checked by rules.
7. **Approve.** A human approves or rejects consequential actions.
8. **Act.** Simulated systems execute: refund, notification, feature flag, ticket.
9. **Verify.** KPIs are measured after the action; the case closes only when targets hold.
10. **Adapt.** New evidence re-opens only the affected steps and shows what changed.

3.2 Golden path for the demo

Seeded synthetic data → run detection → incident → agent investigation with trace → court verdicts → impact → ranked recommendations → approval → simulated action → KPI check on a simulated clock → resolved → jury injects evidence → reassessment → scorecard against hidden truth.

3.3 What makes it different

Feature	What it does	Benefit
Root Cause Court	Tests every hypothesis, decoys included, on timing, segment specificity, telemetry, dose-response and a counter-check	The business fixes the real cause, not the loudest complaint
Customer recovery list	Finds customers with money captured but no order, by transaction state	Customers get their money back before they complain again

Feature	What it does	Benefit
Approval before action	Rollbacks, refunds and messages wait for a human	Control stays with the business
Verified closure	A case closes only when KPIs meet targets for a monitoring window	Recovery is proved, not assumed
Live adaptation	New evidence re-runs only affected tests and shows a diff	Ready for the unseen jury scenario
Self-evaluation	Scores detection, segment, verdict and action eligibility against hidden truth	Claims are measured, not asserted

4. Functional requirements

The system has 36 functional requirements in nine modules; each names its benefit.

4.1 Ingestion

ID	The system shall	Priority	Benefit
FR-1	Accept feedback through `POST /api/v1/feedback`, validate it, store it and return an acknowledgement without waiting for AI analysis	Must	Fast intake from any channel
FR-2	Accept operational events (payments, orders, deliveries, releases) through `POST /api/v1/operational-events`	Must	Root causes can be proved with facts, not opinions
FR-3	Reject duplicate submissions using an idempotency key or source record ID	Must	Counts are not inflated by retries
FR-4	Import feedback from CSV with field mapping, error report and a sample template	Should	Bulk history can be loaded quickly

ID	The system shall	Priority	Benefit
FR-5	Provide simulated import adapters for tickets, app-store reviews, social posts and call transcripts	Should	Shows multi-channel evidence without false claims of live integration
FR-6	Document a Google Forms Apps Script webhook into the feedback API	Could	A real form can feed the system

4.2 Understanding and detection

ID	The system shall	Priority	Benefit
FR-7	Extract category, pain point, component, urgency, financial-loss flag and summary from each feedback text, without inventing IDs or facts	Must	Messy text becomes comparable data
FR-8	Compare current issue volume with a rolling baseline and flag anomalies using count-appropriate statistics, with minimum sample thresholds	Must	Real problems are caught early; noise is ignored
FR-9	Return the baseline, current count, increase and anomaly score with every signal	Must	The alert explains itself

4.3 Clustering

ID	The system shall	Priority	Benefit
FR-10	Cluster feedback with sentence embeddings, so new issues are found without fixed categories	Must	Emerging problems surface on their own
FR-11	Compute cluster membership and counts in code; the LLM may only name clusters	Must	Counts are exact and repeatable

ID	The system shall	Priority	Benefit
FR-12	Merge clusters that describe the same issue across channels	Should	One issue, one case

4.4 Segments

ID	The system shall	Priority	Benefit
FR-13	Compute issue rate, comparison rate, lift and significance by OS, app version, city, payment method, tenure and value tier	Must	The business knows exactly who is hurt
FR-14	Never call a segment affected from raw complaint counts alone	Must	Big segments are not blamed just for being big

4.5 Root Cause Court

ID	The system shall	Priority	Benefit
FR-15	Generate candidate hypotheses from the change log and evidence, including decoys	Must	Alternative explanations are considered
FR-16	Run five tests per hypothesis: timing, segment specificity, telemetry, dose-response, counter-check	Must	Causes are proved, not guessed
FR-17	Return supported, rejected, untested or inconclusive per test; missing data is untested, never proof	Must	Gaps are honest
FR-18	Give each hypothesis a verdict, confidence, supporting evidence, contradicting evidence and missing evidence	Must	The jury sees why a cause won
FR-19	Exclude sentiment from root-cause scoring	Must	Meets the PS guardrail

4.6 Impact

ID	The system shall	Priority	Benefit
FR-20	Calculate affected customers, affected orders, money captured without an order, lost order value and repeat-complaint risk from records	Must	The business sees the cost in rupees
FR-21	Separate payment states: failed before debit, debited then reversed, captured without order, delivered with problem, unknown	Must	Refunds go only to those truly owed
FR-22	Label every figure as measured, estimated or assumed	Must	No figure is overstated

4.7 Interventions, approval and action

ID	The system shall	Priority	Benefit
FR-23	Generate and rank interventions on customer benefit, urgency, cost, time, risk, confidence and reversibility, with evidence references	Must	The most valuable fix goes first
FR-24	Check refund and coupon eligibility with deterministic rules based on transaction state	Must	Nobody is refunded just for complaining
FR-25	Build a customer recovery list with a remedy per eligible customer	Should	Hurt customers are made whole
FR-26	Draft customer messages and support scripts for approval	Should	Consistent, honest communication
FR-27	Hold rollbacks, feature-flag changes, refunds and customer messages for human approval	Must	The business stays in control
FR-28	On rejection, propose the next safe alternative and explain the trade-off	Must	Work continues after a no

ID	The system shall	Priority	Benefit
FR-29	Execute approved actions through simulated adapters and persist each result	Must	Action is shown without real-world risk

4.8 KPI verification and lifecycle

ID	The system shall	Priority	Benefit
FR-30	Attach to each intervention a KPI with baseline, target, window and rollback trigger	Must	Every fix has a measurable goal
FR-31	Mark an incident resolved only when KPIs meet targets for the window after the action, using a simulated clock in the demo	Must	Recovery is proved
FR-32	Move incidents through detected, investigating, awaiting approval, approved, executing, monitoring, resolved, with rejected, escalated and reopened	Must	Status is always clear

4.9 Agent, graph, adaptation and evaluation

ID	The system shall	Priority	Benefit
FR-33	Run a tool-calling agent that plans, calls tools, inspects results and persists every step as a trace	Must	Reasoning is visible and auditable
FR-34	Return the CX issue graph as nodes and edges (signal, issue, segment, cause, intervention, KPI), each linked to evidence	Must	One picture of the whole case
FR-35	Accept injected evidence, re-run only affected tests and return what changed and why	Must	Handles the jury scenario

ID	The system shall	Priority	Benefit
FR-36	Score the run against hidden truth and publish a scorecard	Must	Performance is measured

5. Data requirements

The demo runs on a reproducible synthetic food-delivery world with one real incident, three decoys and a hidden answer key; all of it is labelled synthetic.

5.1 Core entities

Group	Entities
Customers and evidence	customers, feedback, operational_events, product_releases
Investigation	incidents, issue_clusters, incident_evidence, root_cause_hypotheses, investigation_tests
Recovery	recovery_recommendations, approvals, recovery_actions
Verification	kpi_definitions, kpi_measurements
Trace and control	agent_runs, agent_steps, ingestion_jobs, audit_logs

Every evidence row keeps a reference to its source record, so each claim can be traced back.

5.2 Synthetic world

ID	The generator shall	Priority	Benefit
DR-1	Produce about 60 days of customers, orders, payments, deliveries, releases, telemetry and feedback across at least four channels	Must	Realistic multi-source evidence

ID	The generator shall	Priority	Benefit
DR-2	Include enough unaffected customers in every segment for fair rate comparisons	Must	Segment lift is meaningful
DR-3	Use a fixed random seed	Must	Demo numbers repeat exactly
DR-4	Write the hidden truth to a file only the evaluation script can read	Must	Self-evaluation is honest
DR-5	Seed, reset and run-scenario scripts that never touch non-demo data without explicit confirmation	Must	The existing database stays safe

5.3 Planted incident and decoys

Case	Setup	What the system must conclude
Real incident	App v2.3 release breaks UPI payment confirmation for Android 14; some payments captured with no order created	Supported cause, correct segment, money-at-risk calculated
Decoy 1: weather	Rain causes late-delivery complaints on a few days	Separate issue; not linked to payments
Decoy 2: price	A menu price rise draws negative sentiment	Rejected as root cause; sentiment without failure evidence
Decoy 3: seasonal traffic	Exam or festival traffic raises overall volume	Rejected; affects all segments equally

6. AI and LLM requirements

The LLM handles language; code handles every number and every decision rule.

Task	Done by
Extract fields from feedback, name clusters, propose hypotheses, plan tool calls, write explanations and customer messages	LLM
Counting, cluster membership, anomaly scores, segment rates, court tests, impact, eligibility, KPI checks	Python code

ID	The AI service shall	Priority	Benefit
AR-1	Call an open model on Groq through its OpenAI-compatible endpoint, with provider, base URL, model, timeout and retries set in `.env`	Must	Model can be swapped without code changes
AR-2	Validate every LLM output against a JSON schema and fall back safely when invalid	Must	Bad output never reaches the case
AR-3	Link every number in an LLM explanation to the tool output it came from	Must	Every claim is traceable
AR-4	Run the whole golden path with no LLM, using rule extraction, a fixed plan and template text, and mark the case partially analysed	Must	The demo survives a network or Groq failure
AR-5	Budget LLM calls: rules for bulk and seed data, LLM for new records and a few samples per cluster, cached by text hash	Must	Stays inside rate limits
AR-6	Redact email, phone and other identifiers before any external call	Must	Customer privacy
AR-7	Treat feedback text as data; instructions inside it never change agent behaviour or tool permissions	Must	Resistant to prompt injection

ID	The AI service shall	Priority	Benefit
AR-8	Limit agent steps, tool calls, time and tokens per run	Must	Runs stay predictable
AR-9	Use local sentence embeddings, prefetched for offline use, with a TF-IDF fallback	Must	Clustering works without internet
AR-10	Support a local Ollama provider through the same interface	Could	A fully offline option

7. External interfaces

The React client talks only to the Node backend; Node calls the Python AI service with a service token, and Python reads the database through a read-only connection.

ID	The interface shall	Priority	Benefit
IR-1	Serve the public and dashboard API from Node on port 5000 under `/api/v1`, with one response envelope, pagination and documented errors	Must	The client handles every call the same way
IR-2	Serve internal AI endpoints from Python on port 8000, protected by a shared service token and never exposed publicly	Must	AI tools cannot be called from outside
IR-3	Keep Node as the only writer; Python reads through a read-only database user	Must	One system of record, no heavy data transfer
IR-4	Return the issue graph as nodes and edges ready for React Flow	Must	The client draws it directly
IR-5	Offer a mock mode that serves sample responses from day one	Must	The frontend is never blocked
IR-6	Publish `API_CONTRACT.md`, `AI_SERVICE_CONTRACT.md`, OpenAPI and a local setup guide	Must	Integration without guesswork

ID	The interface shall	Priority	Benefit
IR-7	Restrict CORS to the client origin and keep all secrets in `.env`	Must	Safe defaults

7.1 Key backend endpoints

Area	Endpoints
Ingestion	`POST /feedback`, `GET /feedback`, `POST /ingestion/csv`, `GET /ingestion/jobs/:id`, `POST /operational-events`
Signals and dashboard	`GET /dashboard/overview`, `GET /signals`, `POST /signals/:id/investigate`
Incidents	`GET /incidents`, `GET /incidents/:id`, `/graph`, `/evidence`, `/hypotheses`, `/impact`, `/recommendations`, `/agent-trace`, `/timeline`, `/kpis`, `POST /incidents/:id/investigate`, `POST /incidents/:id/reassess`, `POST /incidents/:id/evidence`
Approvals and actions	`GET /approvals`, `POST /approvals/:id/approve`, `POST /approvals/:id/reject`, `POST /incidents/:id/actions/:recommendationId/execute`, `GET /incidents/:id/actions`
Demo	`POST /demo/run-detection`, `POST /demo/advance-clock`, `POST /demo/scenario/:name`, `POST /demo/reset`

7.2 Internal AI endpoints

`GET /health`, `POST /internal/feedback/analyze`, `/signals/detect`, `/issues/cluster`, `/segments/analyze`, `/incidents/investigate`, `/incidents/impact`, `/incidents/recommend`, `/incidents/reassess`, `/agent/run`.

8. Non-functional requirements

These requirements keep customer data safe, keep the business in control, and keep the demo running when something fails. Response times are proposed targets to confirm in the first integration test.

ID	Category	The system shall	Priority
NFR-1	Security	Validate and sanitise all input, verify service tokens, rate-limit public ingestion, hide stack traces	Must
NFR-2	Privacy	Collect minimal personal data, hide contact details from analytics responses, redact identifiers from logs and LLM calls	Must
NFR-3	Safety	Never execute real refunds, messages or production changes; every consequential action needs approval	Must
NFR-4	Integrity	Use transactions for related state changes and make actions idempotent	Must
NFR-5	Reliability	Time out and retry inter-service calls; on AI failure mark the case partially analysed instead of inventing results	Must
NFR-6	Traceability	Record request IDs, agent steps and audit logs for every decision	Must
NFR-7	Performance	Return ingestion acknowledgements in under 1 s and a full investigation in under 60 s on the demo dataset	Should
NFR-8	Honesty	Label all demo data and metrics synthetic; separate measured, estimated and assumed values	Must
NFR-9	Reproducibility	Same seed gives the same data, verdicts and scorecard	Must
NFR-10	Portability	Run on Windows with documented PowerShell and bash commands	Must

9. Jury scenarios and adaptive behaviour

The agent must change its conclusion when the evidence changes, and say what changed and why.

Scenario	Expected behaviour
A hotfix has been deployed	KPIs are re-measured; the case moves to resolved if targets hold, or stays in monitoring
Complaints start on iOS too	Segment analysis re-runs; confidence in the Android-only cause drops; a broader hypothesis is tested
Payment telemetry is missing	The telemetry test becomes untested; confidence falls; nothing is assumed
New support transcripts arrive	They are extracted, clustered into existing issues and added as evidence
The approver rejects the rollback	The agent proposes the next safe intervention and explains the trade-off
Unseen scenario	Evidence goes in through the inject endpoint; the agent re-plans from the missing evidence it now has

Each reassessment returns the new evidence, the hypotheses affected, the tests re-run, the change in verdicts and confidence, and any change to the recommendations.

10. Acceptance and evaluation

Every TCS success measure has a test that can fail; none has run yet, and the scorecard will report measured values only.

ID	TCS success measure	Test	Pass condition
AT-1	Issue detection	Run detection on the seeded data	The planted incident is flagged; detection delay after the v2.3 release is reported in days

ID	TCS success measure	Test	Pass condition
AT-2	Issue detection	Count incidents raised for decoys	Weather becomes a separate low-severity issue; price and seasonal traffic raise no payment incident
AT-3	Segmentation	Compare the reported segment with hidden truth	Android 14 on v2.3 is ranked first by lift
AT-4	Evidence quality	Inspect the court output	The real cause is supported with traceable evidence; all three decoys are rejected or kept separate
AT-5	Evidence quality	Remove telemetry and re-run	The telemetry test is untested, not rejected or supported
AT-6	Impact estimation	Compare money captured without an order with hidden truth	Matches exactly, since it comes from records
AT-7	Intervention relevance	Check eligibility of every proposed refund	No refund for a customer whose payment failed before debit or was reversed
AT-8	Intervention relevance	Reject the rollback	A safe alternative is proposed with its trade-off
AT-9	KPI quality	Approve the rollback, advance the clock	Payment-without-order rate falls below target and holds for the window before resolved
AT-10	KPI quality	Take no action, advance the clock	The incident stays in monitoring or escalates
AT-11	Adaptation	Run each jury scenario in section 9	Verdict, confidence or plan changes as expected, with a diff

ID	TCS success measure	Test	Pass condition
AT-12	No-LLM mode	Set the provider to none and run the golden path	Every step completes; the case is marked partially analysed
AT-13	Hidden truth isolation	Scan agent and tool modules	None reads the hidden truth file

The scorecard reports detection precision and recall, detection delay, segment accuracy, verdict accuracy and action eligibility accuracy.

11. Traceability

Every TCS capability and every PS output maps to requirements and tests.

11.1 TCS core capabilities

Capability	Requirements	Tests
Reason across multiple sources	FR-1, FR-2, FR-5, FR-12	AT-1, AT-4
Plan multiple steps	FR-33	AT-11
Act through tools and systems	FR-29, FR-33	AT-9
Verify against evidence	FR-16 to FR-19	AT-4, AT-5
Adapt	FR-28, FR-35	AT-8, AT-11
Human control	FR-27, FR-28	AT-8
Evaluate with metrics	FR-31, FR-36	AT-9, AT-10, AT-13

11.2 PS expected outcomes

Outcome	Requirements
CX issue graph	FR-34
Affected segment	FR-13, FR-14

Outcome	Requirements
Root-cause hypothesis	FR-15 to FR-19
Evidence	FR-18, AR-3
Impact estimate	FR-20 to FR-22
Prioritised interventions	FR-23 to FR-29
KPI monitoring plan	FR-30, FR-31

12. Constraints, risks and limits

RecoverIQ proves its reasoning in a synthetic world built by the team; it does not prove results on a real company's data, and it says so.

Risk or limit	Effect	Mitigation
About 6 build hours	Features half-built	Golden path first; Should and Could items only after it passes
Synthetic world designed by the team	Easy to look good on it	Hidden truth isolated; decoys planted; scorecard measured, not claimed
Groq rate limits or outage	Agent stalls	Budgeted calls, caching, no-LLM mode
Venue internet fails	Embeddings or LLM unavailable	Prefetched embedding model, TF-IDF fallback, no-LLM mode
Correlation is not causation	A wrong cause could be supported	Five tests, counter-check against unexposed groups, limitations shown
LLM invents a fact	Misleading report	Schema validation; numbers only from tools and linked to them
Prompt text inside feedback	Agent manipulated	Feedback treated as data; tool permissions fixed in code
Existing database	Data loss	Additive migrations only; reset needs explicit confirmation

13. Sources

- TCS Challenge Catalogue: core principles, the 7 capabilities and the jury scenario rule, as shared by the team.
- TCS Challenge 06, Autonomous Customer Experience Recovery: context, required chain, expected outcomes, guardrail and success measures, as shared by the team.
- RecoverIQ backend and AI service implementation specification, written by the team.
- Groq provider guide for the OpenAI-compatible endpoint; check the current model list in the Groq console before choosing a model.